

Oracle Replication: An Introduction

Paul Baumgartel

Introduction

This paper discusses briefly the concepts underlying Oracle Replication, the architecture of a set of replicated objects, and presents a simple example of multimaster replication.

Replication Concepts

Why replicate?

Replication can provide a number of benefits, such as:

- *Availability* of data in the event of server or network failure
- *Performance* gains in distributing activity over multiple servers
- *Integrity* protection when local servers must have copies of master data

Replication types

Oracle offers two modes of replication; the choice of mode will depend on the reason for replication deployment. In some cases, a hybrid is appropriate.

Snapshot replication

Snapshots are known more descriptively as “materialized views” (MVs). An MV is an instantiation of a query, and is updated via an individual batch update process known as a “refresh”. In snapshot replication environments, one database holds the master, and other databases contain the snapshots. Snapshots may be read-only, in which data flows only from the master to the snapshots, or updateable. Oracle propagates the changes made to data in updateable snapshots back to the snapshot’s master site. If other snapshots of the master exist, the changes are applied to them as well.

Multimaster replication

In multimaster replication, replicated objects are peers, and each of them is subject to DML operations. Each database must contain an identical copy of the object; subsets based on queries are not supported. Data changes from each master are then continuously replicated to each of the other masters. This paper discusses multimaster replication.

Deferred transactions

The concept underlying replication is that of deferred transactions. In some types of distributed computing environments, transactions complete at all sites immediately, using a protocol known as two-phase commit. Although Oracle replication may be operated in a synchronous mode, most implementations use an asynchronous approach; deferred transactions at remote sites queue for execution at a later time.

Queuing

Deferred transactions use an interprocess communication mechanism known as message queuing. A transaction queue stores information about operations to be performed at a remote site. Triggers on the master site place transaction information on the queue. A queue processing task runs periodically to service the queue; i.e., to remove messages and forward them to remote sites for processing.

Conflict resolution

Concurrent multimaster transactions present the possibility of simultaneous changes to the same row. While application design can reduce the possibility of such conflicts, it may not be able to eliminate them completely. Oracle provides several built-in conflict resolution methods. If these are insufficient, Oracle allows use of custom (user-written) routines.

Replication administrator

Every replication environment requires a database user to act as the replication administrator. This user acts as an agent for the owners of replicated objects. It is responsible for propagation and receipt of DML changes, and for administrative tasks such as queue purges and building objects required for replication support.

Schema management

DDL applied directly to replicated objects is not replicated. When modifications are to be made to replicated objects, each copy of the object must undergo the same changes to ensure that replication can continue to operate properly. Oracle provides procedures in the DBMS_REPCAT package to apply DDL and replicate it to all copies of a replicated object. The replication group containing the object must be quiesced during this operation.

Quiescing

When replication is enabled, certain operations are prohibited. A replication group can be quiesced (using DBMS_REPCAT.SUSPEND_MASTER_ACTIVITY) to allow maintenance activities to take place. When a group is quiesced, DML operations are not allowed against any of the objects in the group. DBMS_REPCAT.RESUME_MASTER_ACTIVITY takes a replication group out of the quiesced state.

Replication Architecture

Oracle uses internal triggers to capture and store any data changes made to tables that have been defined as replicated. It stores the data changes as remote procedure calls (RPC's) in a deferred transaction queue table for propagation at a predefined time and interval.

Replication groups

In order to participate in replication, an object must belong to a replication group, which is the basic unit of replication activity. Typically a replication group contains logically related objects. Replication groups reside in the schema of the replication administrator account, which creates the groups. Certain replication operations (such as quiesce/dequiesce) apply at the group level only. DBMS_REPCAT.CREATE_MASTER_REPGROUP creates an empty replication group, and DBMS_REPCAT.CREATE_MASTER_REPOBJECT adds objects to the group.

Packages and triggers

The owner of replicated objects owns a replication package for each such object.

DBMS_REPCAT.GENERATE_REPLICATION_SUPPORT creates these packages, as well as “internal” triggers. The triggers fire when DML executes against a replicated object; they call procedures in the packages.

Advanced Queuing

Multimaster replication uses Oracle’s Advanced Queuing facilities. Tables owned by SYSTEM compose the deferred transaction queues and associated administrative objects. Replication facilities are built upon the Advanced Queuing foundation. Troubleshooting replication operation frequently entails inspecting contents of the DEF\$_ tables in the SYSTEM schema.

Database links

Database links are required for replication. In most cases, a single replication administrator schema (often REPADMIN) contains the required database links and replication groups. In addition, the initialization parameter GLOBAL_NAMES, which mandates that a database link name is identical to its target database name, must be set to TRUE.

DBMS jobs

The execution of deferred transactions, as well as the purging of completed transactions and other administrative tasks, takes place from the DBMS_JOB queue. The replication administrator must schedule an instance of DBMS_DEFER_SYS.PUSH for each destination database to run at a desired interval; as the name suggests, it pushes deferred transactions out to destination sites. The administrator must also schedule DBMS_DEFER_SYS.PURGE to purge completed transactions from the queues. Finally the execution of DBMS_REPCAT.GENERATE_REPLICATION_SUPPORT causes DBMS_REPCAT.DO_DEFERRED_REPCAT_ADMIN to run at 10-minute intervals to execute administrative jobs.

Replication API

The Oracle-supplied packages DBMS_REPCAT, DBMS_DEFER_SYS, and DBMS_REPUTIL provide most of the operations required for replication. These may be called directly, or from scripts. In addition, Oracle provides a Replication Manager graphical interface as part of Oracle Enterprise Manager. It generates and executes calls to the same packages.

Case study using multimaster replication

System requirements

The subject system required replication among three databases: a common reference database, a planning database, and an operational database. Common data had to flow to the other two, and planning data had to flow to operations, where replicated data could be modified, but not replicated back to the source. Thus, this is not a true multimaster environment.

Why multimaster was used

The requirements of this system could have been satisfied using snapshot replication, but the designers chose to use multimaster replication for the following reasons:

- Immediacy
- Asynchronism
- Driven from the originating site

Database rebuild requirements

The subject development environment required that test and development databases be rebuilt periodically from a production export. All three databases involved in replication had to be rebuilt concurrently.

Preparation

- Create replication administrator
- Grant privileges
- Register as propagator
- Schedule purge
- Drop schema owners with CASCADE option.
- Re-create schema owners.
- Import objects

Implementation

On each master definition site:

- Since import file contains replicated objects, drop packages named “%\$RP” in replicated object owner schemas.
- Execute DBMS_DEFER_SYS.PURGE (with purge_method = PURGE_METHOD_PRECISE) to clear out deferred transaction records.
- Execute DBMS_REPCAT.CREATE_MASTER_REPGROUP for each replication group on the site.
- Add objects to each replication group via DBMS_REPCAT.CREATE_MASTER_REPOBJECT.
- For each object, build required replication underpinning via DBMS_REPCAT.GENERATE_REPLICATION_SUPPORT.
- For each target database, execute DBMS_DEFER_SYS.SCHEDULE_PUSH.
- For each replication group and each target database, execute DBMS_REPCAT.ADD_MASTER_DATABASE.

Post-implementation

The implementation operations set a sequence of events into motion that can take some time to finish. Progress of the implementation can be monitored by querying the DBA_REPCATLOG view; once this view is empty on all participating databases, replication setup is complete. Database alert logs and trace files contain information about any errors that occur during the implementation process.

The final step before making the replicated objects available to users is to run DBMS_REPCAT.RESUME_MASTER_ACTIVITY for each replication group. This call causes the state of the replication group to change from quiesced to active.

Appendix

The following script sets up replication, using one replication group, from one database to two others. It assumes that the replication administrator account exists, with the necessary privileges.

```
-----  
-- CC REPGROUP  
-----  
begin  
  dbms_repcat.create_master_repgroup('CC_REP_GROUP');  
  
  dbms_repcat.create_master_repobject('A010CC', 'TACNTYP_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TALTASN_CC', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TASYSVC_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TBCLTNR_CC', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TBCLTYP_CC', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TCCMSLG', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TCCYTB_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TCTCYPR_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TCTMPST_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TCTMSMN_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');  
  dbms_repcat.create_master_repobject('A010CC', 'TDMAPRT_CC_A0', 'TABLE',  
                                       GNAME=>'CC_REP_GROUP');
```

```

dbms_repcat.create_master_repobject('A010CC', 'TDOWTBL_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TDITYTYP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TGPOCSF_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'THLCYIA_CC', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'THLOGIA_CC', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TINFATH_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TINFSYS_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TINSRCT_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TLBLTYP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TMKGTPT_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TMVMTP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TOPNTYP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPKACTP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPKEXRE_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPKGLRL_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPOLDV1_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPOLDV2_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPOLDV3_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPRIASN_CC', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TPSVCTP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TSHPTYP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TSOPKCL_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TSRRXTP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TSRTTYP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TSVCCDE_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');
dbms_repcat.create_master_repobject('A010CC', 'TULDTP_CC_A0', 'TABLE',
                                     GNAME=>'CC_REP_GROUP');

dbms_repcat.generate_replication_support('A010CC', 'TACNTYP_CC_A0', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TALTASN_CC', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TASYSVC_CC_A0', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TBCLTNR_CC', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TBCLTYP_CC', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TCCMSLG', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TCCYTB_CC_A0', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TCTCYPR_CC_A0', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TCTMPST_CC_A0', 'TABLE');
dbms_repcat.generate_replication_support('A010CC', 'TCTMSMN_CC_A0', 'TABLE');

```

```

dbms_repcat.generate_replication_support('A010CC','TDMAPRT_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TDOWTBL_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TDTYTYP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TGPOCSF_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','THLCYIA_CC','TABLE');
dbms_repcat.generate_replication_support('A010CC','THLOGIA_CC','TABLE');
dbms_repcat.generate_replication_support('A010CC','TINFATH_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TINFSYS_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TINSRCT_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TLBLTYP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TMKGTPT_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TMVMTP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TOPNTYP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPKACTP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPKEXRE_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPKGLRL_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPOLDV1_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPOLDV2_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPOLDV3_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPRIASN_CC','TABLE');
dbms_repcat.generate_replication_support('A010CC','TPSVCTP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TSHPTYP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TSOPKCL_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TSRRXTP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TSRTTYP_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TSVCCDE_CC_A0','TABLE');
dbms_repcat.generate_replication_support('A010CC','TULDTP_CC_A0','TABLE');
end;
/

prompt
prompt When prompted, enter name of target 650 database (e.g., A650D.TRALFAZ.COM)
prompt

    dbms_defer_sys.schedule_push(destination=>'&&target_650_db',
                                interval =>'sysdate + 1/2880',
                                next_date=>sysdate);

prompt
prompt When prompted, enter name of target 657 database (e.g., A657D.TRALFAZ.COM)
prompt

    dbms_defer_sys.schedule_push(destination=>'&&target_657_db',
                                interval =>'sysdate + 1/2880',
                                next_date=>sysdate);

    dbms_repcat.add_master_database('CC_REP_GROUP', '&&target_650_db',
                                    use_existing_objects=>TRUE, copy_rows=>FALSE);
    dbms_repcat.add_master_database('CC_REP_GROUP', '&&target_657_db',
                                    use_existing_objects=>TRUE, copy_rows=>FALSE);
    dbms_defer_sys.schedule_purge(purge_method=>2, next_date=>sysdate,
                                interval=>'sysdate + 1/24');

exit

```